

Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** My Nutrition Partner
- **Team Members:** Single-developer full-stack engineering implementation.
(P. NARNDRANATH)

2. Project Overview

- **Purpose:** The application provides a lightweight web interface for active individuals to monitor daily macronutrient thresholds and design caloric strategy frameworks depending on specific physical body types and fitness tracks.
- **Features:**
 1. Secure split authentication window supporting instant Sign In and Sign Up toggles.
 2. Dynamic consumption logging feed tracks for Calories, Protein, Carbohydrates, and Fats.
 3. Real-time progress circular percentage widgets matching defined total target boundaries.
 4. Dynamic database-driven personalized diet blueprint matrix calculations.
 5. Full Cloud MongoDB Atlas interaction mappings.

3. Architecture

- **Frontend:** Built as a single-page reactive application using React v18+ managed by the Vite build tool system. Interfaces utilize localized state management engines integrated directly with responsive Bootstrap v5 CSS utility wrappers for visual components.
- **Backend:** A Node.js runtime container running the Express.js framework to serve RESTful routing architectures and intercept external client API calls.
- **Database:** A MongoDB NoSQL distributed cloud document schema layer handled via Mongoose Object Data Modeling (ODM). It contains two separate dedicated collections: profiles (storing unique physical metric indexes) and logs (storing historic tracked consumption feed parameters).

4. Setup Instructions

- **Prerequisites:** Node.js runtime environment installed locally and An active MongoDB Cloud Atlas connection cluster string URI.
- **Installation:**
 1. Clone & Open Root Project:
cd NutritonAssistant-MERN
 2. Install Environment Core Dependencies:
npm install
 3. Configure Environment Keys:
Create a file named .env in the absolute root folder and define your ports and Atlas connection addresses:
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@c

```
cluster0.mongodb.net/nutritionDB?retryWrites=true&w=majority
```

4. Install Frontend Build Dependencies:

```
cd public
npm install
```

5. Folder Structure:

- The app layout operates under an upgraded MVC system where the frontend application builds and compiles natively inside the public directory:

```
NutritonAssistant-MERN/
├── server.js      # Express application pipeline server setup
├── .env           # Cloud database configuration credentials
├── package.json  # Main system server configuration keys
├── public/       # React UI frontend project folder
│   ├── index.html # Base DOM viewport root mount file
│   ├── vite.config.js # Vite compilation configuration settings
│   ├── package.json # Frontend asset module configuration
│   └── src/
│       ├── App.jsx # Core app logic containing tabs and layout routing
│       ├── main.jsx # Document mount initialization hook
│       ├── App.css # Reset clean custom style configurations
│       └── index.css # Layout standard alignment specifications
```

6. Running the Application

- To run the full-stack environment successfully, you must open two separate terminal prompts in your working editor:

1. Start the Backend Environment

From the absolute main directory terminal window (NutritonAssistant-MERN), execute:

```
node server.js
```

Expected console log message: 🔥 MongoDB Database Connected Successfully to Cloud Atlas!

2. Start the Frontend Interface

Open a second parallel terminal window tab, navigate into the public application workspace folder, and initialize the development compilation server:

```
cd public
npm run dev
```

Expected output: The app launches local tracking metrics actively via standard development ports at <http://localhost:5173/>.

7. API Documentation

- The backend engine unifies data streams across five core REST endpoints:

1. Fetch Food Consumption Log Entries:

Method/Route: Get/api/logs

2. Log a New Meal Item:

Method/Routes: Post/api/logs

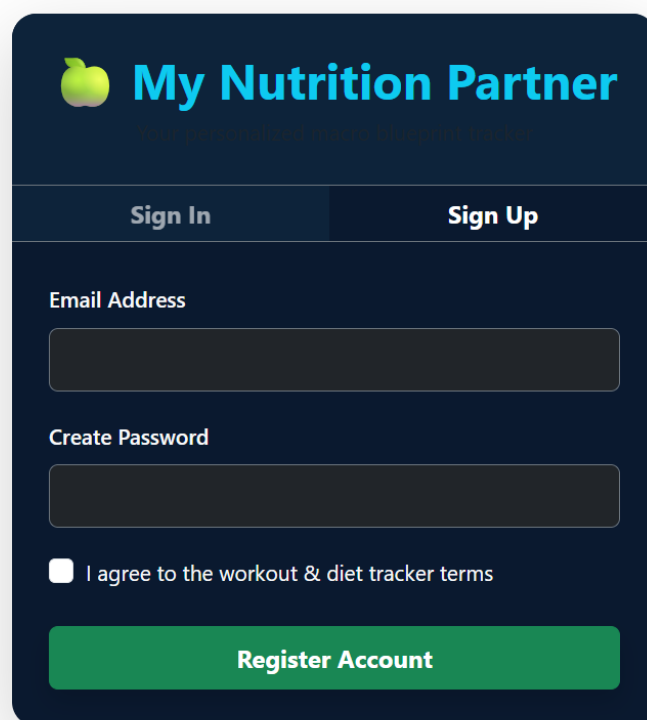
3. Remove Log Entry From Cluster:
Method/Routes: Delete\api\logs\id
4. Fetch User Profile Setup Configurations:
Methods/Routes: Get\api\logs\Profile
5. Save/Update Profile Settings:
Method / Route: POST /api/logs/profile

8. Authentication

- **Mechanism:** Handled via clean user session state flows
- **Operation:** The application incorporates a dynamic dual auth design. When a user inputs their target tracking email address inside the custom dark blue dashboard panel card, the app binds the tracking metrics context globally. Signing out immediately clears all profile caches, component logs, and active navigation configurations to return to a fresh landing screen state.

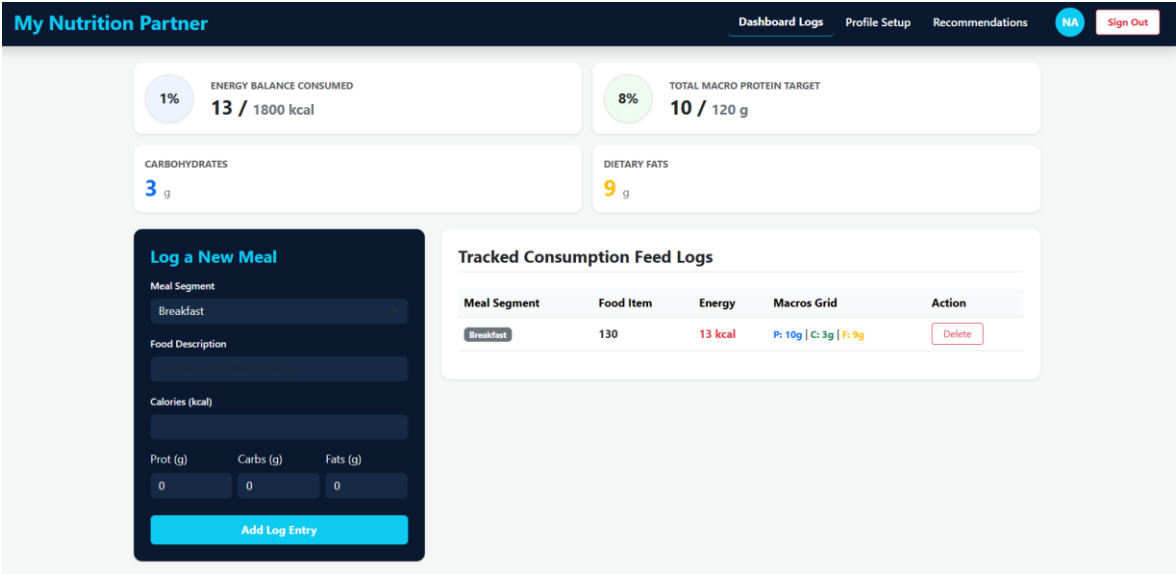
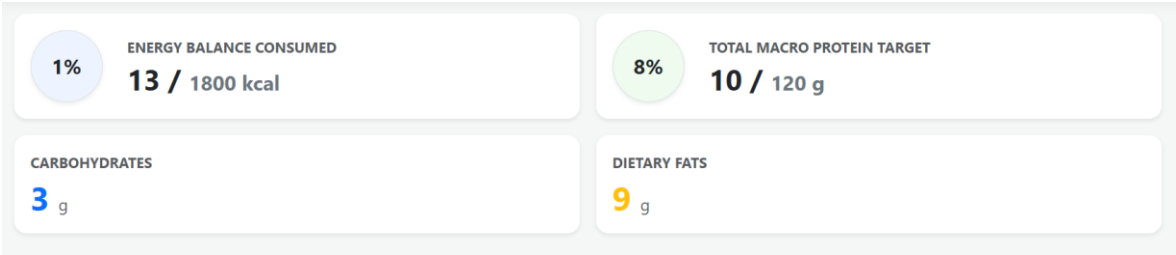
9. User Interface: The portal's visual architecture provides clean layouts styled for high scannability.

- ☐ **Authentication Hub:** Split layout featuring matching Sign In and Sign Up interactive navigation triggers set against deep navy blue elements.



The image shows a dark blue dashboard panel for 'My Nutrition Partner'. At the top left is a green apple icon. To its right is the text 'My Nutrition Partner' in a light blue font. Below this is a horizontal split layout with two buttons: 'Sign In' on the left and 'Sign Up' on the right, both in white text. Under the 'Sign Up' button, there is a form with the following elements: a label 'Email Address' above a dark grey input field; a label 'Create Password' above another dark grey input field; a checkbox with the text 'I agree to the workout & diet tracker terms'; and a large green button at the bottom with the text 'Register Account' in white.

- ☐ **Dashboard Tracker view:** Features dynamic structural widgets displaying exact consumption percentages alongside structural forms to update meal counts cleanly.



Update Physical Configurations

Full Profile Name

Narendra

Age Index (Years)

20

Bodyweight (kg)

70

Height (cm)

124

Select Target Blueprint Strategy

Weight Loss

Save Configuration Plan

- Profile Control Configurator: Clean grid inputs allowing data tracking customization for targeted caloric requirements.

The screenshot shows a web application titled "My Nutrition Partner". The navigation bar includes "Dashboard Logs", "Profile Setup", "Recommendations" (which is active), a user profile icon labeled "NA", and a "Sign Out" button. The main content area is titled "Nutritional Blueprint Matrix" and indicates it is for "Narendra (Weight Loss)". It features a "Calculated Caloric Level Target" of a deficit of ~1,500 - 1,800 kcal/day. Below this, it lists "Macro Matrix Routine Strategies" and provides specific meal blueprints for breakfast and lunch.

My Nutrition Partner Dashboard Logs Profile Setup **Recommendations** NA Sign Out

Nutritional Blueprint Matrix

Target configuration dynamic logic strategies matching: **Narendra (Weight Loss)**

Calculated Caloric Level Target
Deficit (Target: ~1,500 - 1,800 kcal/day)

Macro Matrix Routine Strategies
Maintain a rigid clean caloric deficit window. Emphasize dense protein isolate structures and green fibrous vegetables while omitting refined dynamic sugars.

Breakfast Blueprints

- 3 Egg whites scrambled with baby spinach, tomatoes and mushrooms
- Sprouted green lentils gram salad topped with olive oil and squeezed lime
- Oatmeal prepared with warm water, standard almond milk, and raw berries

Lunch Configurations

- 150g Boiled fish or baked organic tofu strips with a large leaf salad bowl
- Chana chickpea salad tossed with chopped cucumber, radish, and raw onions
- Clear hot vegetable broth alongside small measured brown rice frames

- Diet Blueprint panel: An analytical tab split into structured meal category cards showcasing morning, lunch, evening, and bedtime meal configurations aligned with your workout track.

The screenshot shows a "Log a New Meal" form. It has a dark blue background with white and light blue text and input fields. The form includes a "Meal Segment" dropdown menu currently set to "Breakfast", a "Food Description" text input field with a placeholder "e.g., Rice & Grilled Chicken", a "Calories (kcal)" text input field, and three input fields for "Prot (g)", "Carbs (g)", and "Fats (g)", each with a value of "0". At the bottom is a large blue button labeled "Add Log Entry".

Log a New Meal

Meal Segment

Breakfast

Food Description

e.g., Rice & Grilled Chicken

Calories (kcal)

Prot (g) Carbs (g) Fats (g)

0 0 0

Add Log Entry

10. Testing:

Strategy: Tested via End-to-End (E2E) component tracking validation methods.

Execution Parameters: Route integrations, database updates, input data sanitization formats, and conditional macro target engines were verified across extreme validation edge cases using Chrome Developer Tools framework consoles to achieve a zero ESLint fault metric signature.

11. Demo:

https://drive.google.com/file/d/1k6pbpBUlzVRhLmiYNebSpOsQHlvCX9NK/view?usp=drive_link

12. Known Issues:

- DNS Resolution Over Hotspots: Local Node.js configurations can sometimes encounter cluster connection timeouts on restricted mobile network hotspots. The application explicitly uses custom DNS bindings (8.8.8.8) inside server.js to circumvent this network constraint.

13. Future Enhancements:

- BCrypt Integration: Encrypt password keys with cryptographic salts during transmission to database cluster systems.
- JSON Web Token (JWT) Handling: Upgrade state-based sign-in structures to persistent JWT cookie storage schemes.
- Automated Barcode Analysis: Integrate device cameras to decode nutritional data from product barcodes instantly.

